Riesgo de sobrecalentamiento de un chip con carga de trabajo incierta: elementos finitos y cuantificación probabilística para el diseño de disipadores

Nancy Burgos, Carlos Arroyave, David Restrepo

Escuela de Ciencias Aplicadas e Ingeniería, Universidad EAFIT, Medellín, Colombia

Resumen

El disipador de un procesador se diseña antes de conocer la carga de trabajo, de modo que el mapa de potencia que debe evacuar es incierto. En este trabajo se estima la probabilidad de que la temperatura máxima de un chip supere su límite cuando la activación de sus bloques funcionales es aleatoria y espacialmente correlacionada. La temperatura se modela con la ecuación de Helmholtz modificada en dos dimensiones y se resuelve con un programa propio de elementos finitos que admite geometrías arbitrarias, varios materiales y condiciones de frontera mixtas, verificado con dos soluciones manufacturadas. Las activaciones se generan con una cópula gaussiana; gracias a la linealidad del problema, la probabilidad de falla se calcula de forma exacta y se contrasta con Monte Carlo. Con la misma capacidad total de enfriamiento, la probabilidad de falla de cuatro estrategias de disipación varía entre 0,04 y 0,75. Diseñar según la potencia media da casi la misma temperatura esperada que diseñar según la potencia pico, pero siete veces más probabilidad de falla, y la correlación de la carga multiplica por más de treinta el riesgo del mejor diseño. Un análisis determinista no basta para comparar estrategias de disipación.

Palabras clave: elementos finitos, gestión térmica de chips, cuantificación de incertidumbre, Monte Carlo, cópula gaussiana, soluciones manufacturadas

1. Introducción

La densidad de potencia de los circuitos integrados ha crecido más rápido que la capacidad de evacuar el calor que producen. Desde mediados de la década de 2000 la temperatura dejó de ser un problema exclusivo del empaquetado y pasó a condicionar la arquitectura misma del procesador [1, 2]. La temperatura máxima de unión de un procesador comercial suele estar entre 100 °C y 110 °C [3]; por encima de ese límite el chip reduce su frecuencia o se apaga, y la operación prolongada a alta temperatura acelera los mecanismos de degradación [2]. Una consecuencia extrema de esta restricción es el llamado *silicio oscuro*: en los nodos tecnológicos recientes no es posible mantener encendidos todos los transistores del chip al mismo tiempo sin exceder el presupuesto de potencia, de modo que en cada instante solo una fracción de los bloques está activa [4].

Esto tiene una implicación directa para el diseño del disipador. El disipador se diseña una sola vez, antes de saber qué programas ejecutará el chip; en cambio, qué bloques están encendidos en cada momento depende del programa, del sistema operativo y del usuario. El mapa de potencia que el disipador debe evacuar no es un dato sino una variable aleatoria. Las dos alternativas deterministas habituales tienen limitaciones claras. Diseñar para el peor caso (todos los bloques encendidos) sobredimensiona el sistema y, como se mostrará, no permite distinguir entre diseños que en la práctica tienen riesgos muy distintos. Diseñar para la potencia media subestima los bloques que consumen mucho pero se usan poco, y esconde los puntos calientes que aparecen cuando varios bloques vecinos se activan juntos.

La modelación térmica de chips en la etapa temprana de diseño suele hacerse con modelos compactos de red térmica como HotSpot [5], que representan el chip como una red de resistencias y capacitancias con un camino vertical hacia el disipador. Esos modelos se evalúan normalmente con trazas de potencia concretas. En este trabajo se adopta un enfoque complementario: se resuelve la ecuación de conducción de calor con el método de elementos finitos y se trata la activación de los bloques como un vector aleatorio con dependencia espacial, para estimar la *probabilidad* de que la temperatura máxima supere el límite. Esa probabilidad se usa como criterio para comparar estrategias de disipación con la misma capacidad total de enfriamiento.

Los objetivos del proyecto son: (i) implementar un solver de elementos finitos general, capaz de resolver el problema en geometrías arbitrarias, con varios materiales y condiciones de frontera mixtas a partir de archivos de entrada; (ii) verificar el solver con soluciones manufacturadas; (iii) modelar las fuentes de calor como bloques con activación aleatoria y correlacionada; (iv) estimar la probabilidad de falla térmica de distintas estrategias de disipación; y (v) analizar su sensibilidad a la correlación espacial y al nivel de carga.

El artículo está organizado así. La Sección 2 presenta el modelo físico y probabilístico, la discretización por elementos finitos, la implementación y los métodos para estimar la probabilidad de falla. La Sección 3 verifica el solver y presenta y analiza los resultados del estudio del chip, y la Sección 4 resume las conclusiones y limitaciones.

2. Metodología

Esta sección presenta primero el modelo matemático del problema, físico y probabilístico, y después los métodos numéricos con los que se resuelve.

2.1. Modelo térmico bidimensional

El chip es una lámina de silicio de lado $L = 10\,\text{mm}$ y espesor $H = 0{,}5\,\text{mm}$. Su cara inferior contiene los transistores, que generan calor, y su cara superior está en contacto con el disipador a través de un material de interfaz. En estado estacionario la temperatura cumple $-\nabla \cdot (k\nabla T) = q$ en el volumen del chip. Como la lámina es delgada, la temperatura casi no varía en el espesor: el número de Biot asociado a la cara superior es $\text{Bi} = h_{\text{sup}}H/k \approx 0{,}07 < 0{,}1$ con los valores de la Tabla 1, el criterio usual para despreciar los gradientes internos [6]. Integrando la ecuación en el espesor y modelando el flujo por la cara superior con un coeficiente de transferencia $h_{\text{sup}}(\boldsymbol{x})$ que resume todo el camino térmico hasta el aire (interfaz, base del disipador y aletas), se obtiene el problema bidimensional

$$-\nabla \cdot (k\nabla T) + \beta(\boldsymbol{x})\,(T - T_{\text{amb}}) = q(\boldsymbol{x}) \quad \text{en } \Omega = (0, L)^2, \qquad \beta(\boldsymbol{x}) = \frac{h_{\text{sup}}(\boldsymbol{x})}{H}, \tag{1}$$

donde T es la temperatura promediada en el espesor, k la conductividad del silicio, q la potencia volumétrica disipada y T_{amb} la temperatura del aire dentro del equipo. Por los cuatro bordes laterales el chip intercambia calor por convección natural,

$$-k\,\frac{\partial T}{\partial n} = h_b\,(T - T_{\text{amb}}) \quad \text{en } \partial\Omega. \tag{2}$$

Escribiendo $f = q + \beta T_{\text{amb}}$, la ecuación (1) toma la forma $-\nabla \cdot (k\nabla T) + \beta T = f$, que es la ecuación de *Helmholtz modificada* (o de Poisson apantallada): la ecuación de Helmholtz con número de onda imaginario. A diferencia de la ecuación de Poisson, el término βT hace que el efecto de una fuente decaiga con una longitud característica $\lambda = \sqrt{k/\beta} = \sqrt{kH/h_{\text{sup}}}$. Con los valores de la Tabla 1, $\lambda \approx 1{,}9\,\text{mm}$, del orden del tamaño de un bloque: el calor de un bloque se evacúa principalmente hacia arriba, cerca de donde se genera, y la distribución espacial de h_{sup} importa. Esa es la razón física de que la ubicación del disipador sea una variable de diseño relevante.

Tabla 1: Parámetros físicos del modelo.

Símbolo	Descripción	Valor	Fuente
L, H	Lado y espesor del chip	10 mm, 0,5 mm	valores típicos
k	Conductividad del silicio (300 K)	148 W/(m K)	[7]
$\bar{h}_{\text{sup}}$	Coeficiente medio por la cara superior	2×10^4 W/(m^2 K)	$R_{ja} \approx 0{,}5$ K/W
h_b	Convección en los bordes	10 W/(m^2 K)	convección natural [6]
T_{amb}	Temperatura del aire interno	40 °C	supuesto
T_{crit}	Temperatura máxima de unión	100 °C	[3]

2.2. Plano del chip y modelo de fuentes

El chip contiene $N_b = 8$ bloques funcionales $\Omega_i \subset \Omega$: cuatro núcleos, una caché de último nivel, una GPU, un controlador de memoria y un bloque de entrada/salida (Tabla 2 y Figura 1, arriba a la izquierda). Cada bloque está encendido o apagado, y cuando está encendido disipa una potencia P_i repartida uniformemente en su volumen:

$$q(\boldsymbol{x}; \boldsymbol{A}) = \sum_{i=1}^{N_b} A_i \, q_i \, \chi_{\Omega_i}(\boldsymbol{x}), \qquad q_i = \frac{P_i}{|\Omega_i| \, H}, \qquad A_i \in \{0, 1\}, \tag{3}$$

donde χ_{Ω_i} es la función indicadora del bloque. Las potencias y las probabilidades de activación $p_i = P(A_i = 1)$ de la Tabla 2 son supuestos representativos de un procesador de escritorio: los núcleos tienen la mayor densidad de potencia, la GPU consume mucho pero se usa poco y la caché está activa casi siempre. Con todos los bloques encendidos el chip disipa 106 W.

Tabla 2: Plano de bloques del chip. p_i es la probabilidad de que el bloque esté activo.

Bloque	Tamaño [mm]	P_i [W]	Densidad [W/mm^2]	p_i
Núcleos 1–4	2×2 (c/u)	12 (c/u)	3,00	0,5
Caché L3	9.2×1.6	8	0,54	0,7
GPU	5.6×4.4	40	1,62	0,3
Controlador de memoria	3.2×2.0	6	0,94	0,6
Entrada/salida	3.2×2.0	4	0,63	0,4

Las activaciones no son independientes: tareas que usan varios núcleos o datos compartidos tienden a activar bloques cercanos al mismo tiempo. La dependencia se modela con una cópula gaussiana, siguiendo el método NORTA (*NORmal To Anything*) [8]. Se toma un vector latente $\boldsymbol{Z} \sim \mathcal{N}(\boldsymbol{0}, \Sigma)$ con correlación exponencial en la distancia d_{ij} entre los centros de los bloques,

$$\Sigma_{ij} = \exp\!\left(-\frac{d_{ij}}{\ell}\right), \qquad A_i = \mathbf{1}\{Z_i > c_i\}, \qquad c_i = \Phi^{-1}(1 - p_i), \tag{4}$$

donde ℓ es la longitud de correlación y Φ la función de distribución normal estándar. Así cada A_i tiene exactamente la probabilidad marginal p_i, y bloques más cercanos están más correlacionados. El caso $\ell \to 0$ corresponde a bloques independientes, y $\ell \to \infty$ a activaciones totalmente dependientes, en las que los bloques se encienden en un orden fijo, de mayor a menor p_i. La correlación entre las variables binarias es menor que la latente:

$$\rho^{A}_{ij} = \frac{\Phi_2(-c_i, -c_j; \Sigma_{ij}) - p_i p_j}{\sqrt{p_i(1 - p_i)\, p_j(1 - p_j)}}, \tag{5}$$

donde $\Phi_2(\cdot, \cdot; \rho)$ es la distribución normal bivariada con correlación ρ. Para dos núcleos vecinos con $\ell = 3$ mm, la correlación latente es 0,449 y la de las activaciones es 0,297.

2.3. *Probabilidad de falla y estrategias de disipación*

Se dice que el chip falla térmicamente si su temperatura máxima supera T_{crit}. La medida de riesgo es

$$P_{\text{falla}} = P(T_{\text{máx}}(\boldsymbol{A}) > T_{\text{crit}})\,, \qquad T_{\text{máx}}(\boldsymbol{A}) = \max_{\boldsymbol{x}\in\Omega} T(\boldsymbol{x};\boldsymbol{A}). \tag{6}$$

Una estrategia de disipación es una función $h_{\text{sup}}(\boldsymbol{x})$. Para que la comparación sea justa, todas las estrategias tienen la misma capacidad total de enfriamiento, $\int_\Omega h_{\text{sup}}\,\mathrm{d}A = \bar{h}_{\text{sup}}\,L^2$, de modo que solo cambia *dónde* se enfría y no *cuánto*. Se estudian cuatro estrategias (Figura 1):

- **Uniforme:** $h_{\text{sup}} = \bar{h}_{\text{sup}}$, el disipador estándar.
- **Centrado:** una placa de $6 \times 6\,\text{mm}$ en el centro con coeficiente alto y $h_{\text{sup}} = 0{,}5\,\bar{h}_{\text{sup}}$ fuera de ella. Representa un diseño ingenuo que enfría el centro geométrico del chip.
- **Según potencia media:** $h_{\text{sup}} = 0{,}5\,\bar{h}_{\text{sup}} + c\sum_i P_i p_i\,\chi_{\Omega_i}/|\Omega_i|$, es decir, enfriamiento adicional proporcional a la densidad de potencia *esperada*.
- **Según potencia pico:** igual, pero proporcional a la densidad de potencia con el bloque encendido, $P_i/|\Omega_i|$, sin ponderar por p_i.

En los tres diseños no uniformes la constante se elige para conservar la integral de h_{sup}.

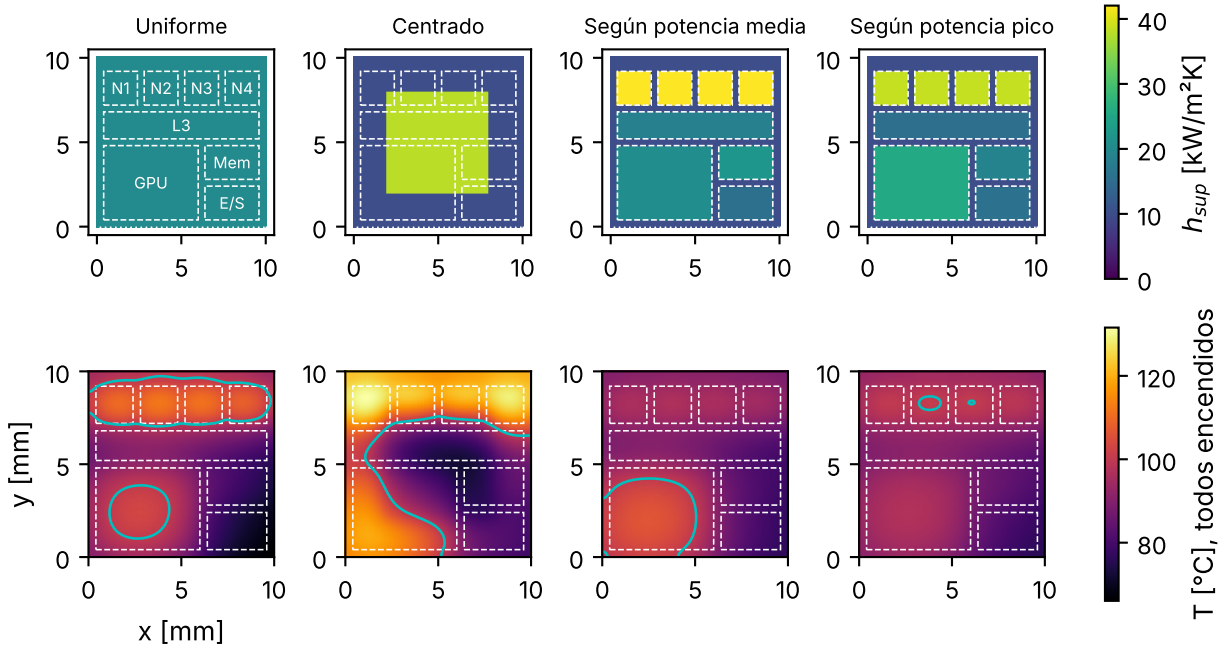


Figura 1: Arriba: coeficiente h_{sup} de las cuatro estrategias, todas con la misma capacidad total de enfriamiento; las líneas discontinuas marcan los bloques (N1–N4: núcleos, L3: caché, Mem: controlador de memoria, E/S: entrada/salida). Abajo: temperatura con todos los bloques encendidos; la línea cian es la isoterma $T_{\text{crit}} = 100\,°\text{C}$.

2.4. *Formulación débil y discretización*

El solver resuelve el problema general

$$\begin{aligned} -\nabla\cdot(k\nabla T)+\beta\,(T-T_{\mathrm{ref}}) &= q && \text{en } \Omega,\\ T &= T_D && \text{en } \Gamma_D,\\ k\,\partial_n T &= g && \text{en } \Gamma_N,\\ -k\,\partial_n T &= h\,(T-T_\infty) && \text{en } \Gamma_R, \end{aligned} \tag{7}$$

donde k, β, T_{ref} y q pueden tomar valores distintos en cada material y $\partial\Omega=\Gamma_D\cup\Gamma_N\cup\Gamma_R$. El problema del chip (1)–(2) es el caso particular con $\Gamma_R=\partial\Omega$. Si $V=\{v\in H^1(\Omega): v|_{\Gamma_D}=0\}$, multiplicando por $v\in V$ e integrando por partes se obtiene la formulación débil: hallar T, con $T=T_D$ en Γ_D, tal que

$$\int_\Omega k\nabla T\cdot\nabla v\,\mathrm{d}\Omega+\int_\Omega \beta\,Tv\,\mathrm{d}\Omega+\int_{\Gamma_R} h\,Tv\,\mathrm{d}s=\int_\Omega (q+\beta T_{\mathrm{ref}})v\,\mathrm{d}\Omega+\int_{\Gamma_R} h\,T_\infty v\,\mathrm{d}s+\int_{\Gamma_N} g\,v\,\mathrm{d}s \qquad \forall v\in V. \tag{8}$$

Las condiciones de Neumann y Robin son naturales: aparecen en la forma débil y no hay que imponerlas sobre el espacio. Como $k>0$, $\beta\geq 0$ y $h\geq 0$, la forma bilineal es simétrica y coerciva siempre que haya alguna frontera de Dirichlet, de Robin con $h>0$ o un término $\beta>0$; esto garantiza solución única [9].

El dominio se aproxima con una malla de triángulos y T con funciones lineales a trozos (elementos P1), $T_h=\sum_j T_j\varphi_j$. Al tomar $v=\varphi_i$ se obtiene el sistema lineal

$$(\mathbf{K}+\mathbf{M}_\beta+\mathbf{R})\,\mathbf{T}=\mathbf{F}+\mathbf{G}, \tag{9}$$

con las matrices elementales, en un triángulo e de área A_e y una arista de frontera de longitud L_s,

$$\mathbf{K}_e=k_e A_e\,\mathbf{B}_e^{\top}\mathbf{B}_e, \qquad \mathbf{M}_{\beta,e}=\frac{\beta_e A_e}{12}\begin{pmatrix}2&1&1\\1&2&1\\1&1&2\end{pmatrix}, \qquad \mathbf{R}_s=\frac{h_s L_s}{6}\begin{pmatrix}2&1\\1&2\end{pmatrix}, \tag{10}$$

donde $\mathbf{B}_e$ contiene los gradientes constantes de las funciones de forma. Las matrices se obtienen simbólicamente con SymPy [10], mapeando cada triángulo al de referencia, y se convierten en funciones vectorizadas que las evalúan en todos los elementos a la vez. El vector $\mathbf{G}$ reúne las contribuciones de Robin, Neumann y del término βT_{ref}. Para cargas que dependen de la posición, $\mathbf{F}$ se integra con la regla de los puntos medios de las aristas, exacta para polinomios de grado dos. Las condiciones de Dirichlet se imponen por levantamiento: separando los nodos libres L de los fijos D, se resuelve $\mathbf{A}_{LL}\mathbf{T}_L=\mathbf{b}_L-\mathbf{A}_{LD}\mathbf{T}_D$.

La matriz se ensambla en formato disperso y se factoriza una sola vez con una descomposición LU dispersa (SuperLU, a través de SciPy [11]). En la malla del chip de 100×100 celdas (10.201 nodos y 20.000 triángulos), el ensamblaje y la factorización toman 0,06 s y cada solución posterior 1,6 ms.

2.5. *Implementación y generalidad*

El programa está escrito en Python. Son implementación propia: el lector de mallas, el ensamblaje, las matrices elementales, la imposición de las tres condiciones de frontera, el modelo de fuentes, el muestreo NORTA y el cálculo de probabilidades. Se usan como herramientas externas Gmsh [12] para generar mallas no estructuradas, NumPy [13] para el cálculo vectorial, SciPy [11] para la factorización dispersa y las probabilidades normales multivariadas, SymPy para las matrices elementales y las soluciones manufacturadas, y Matplotlib para las figuras.

Un problema se describe con dos archivos. El primero es una malla de Gmsh (formato MSH 2.2) con *grupos físicos* que nombran las regiones y los tramos de frontera. El segundo es un archivo JSON que asigna a cada región sus propiedades (k, q, β, T_{ref}) y a cada tramo de frontera su tipo de condición y sus datos. Los datos pueden ser constantes o expresiones en x e y. El lector reorienta automáticamente los triángulos en sentido antihorario para que el jacobiano sea positivo. Así, un problema nuevo se resuelve cambiando los archivos de entrada, sin modificar el código; la Sección 3.1 lo ilustra con dos geometrías distintas del chip. El código, las pruebas automáticas y los archivos de entrada están disponibles en el repositorio del proyecto (véase la sección *Disponibilidad del código*).

2.6. Superposición y cálculo exacto de la probabilidad de falla

La EDP es lineal y, para un diseño dado, la matriz de (9) no depende de $\boldsymbol{A}$. Por superposición,

$$\mathbf{T}(\boldsymbol{A}) = \mathbf{T}_0 + \sum_{i=1}^{N_b} A_i \, \mathbf{U}_i, \tag{11}$$

donde $\mathbf{T}_0$ es la solución sin fuentes y $\mathbf{U}_i$ la respuesta al bloque i encendido con datos de frontera homogéneos. Basta entonces resolver $N_b + 1 = 9$ sistemas por diseño, y cada realización se reduce a una suma de vectores. Como T_h es lineal en cada triángulo, su máximo se alcanza en un nodo, de modo que $T_{\text{máx}}$ se evalúa exactamente sobre los valores nodales.

Con bloques binarios solo hay $2^{N_b} = 256$ configuraciones posibles $\boldsymbol{a}$. Por tanto, $T_{\text{máx}}(\boldsymbol{a})$ se calcula para todas ellas y la probabilidad de falla se obtiene de forma exacta como

$$P_{\text{falla}} = \sum_{\boldsymbol{a} \in \{0,1\}^{N_b}} \pi(\boldsymbol{a}) \, \mathbf{1}\{T_{\text{máx}}(\boldsymbol{a}) > T_{\text{crit}}\}, \qquad \pi(\boldsymbol{a}) = P(\boldsymbol{A} = \boldsymbol{a}). \tag{12}$$

Cada probabilidad $\pi(\boldsymbol{a})$ es la probabilidad de que $\boldsymbol{Z}$ caiga en un ortante: $Z_i > c_i$ si $a_i = 1$ y $Z_i \le c_i$ si $a_i = 0$. Cambiando el signo de las componentes encendidas, se reduce a una función de distribución normal multivariada, que se evalúa con el algoritmo cuasi-Monte Carlo de Genz [14] con tolerancia absoluta 10^{-6}. Las 256 probabilidades toman 1,4 s. De la misma forma se obtienen sin error de muestreo el campo medio $\mathbb{E}[\mathbf{T}] = \mathbf{T}_0 + \sum_i p_i \mathbf{U}_i$, su desviación estándar y la probabilidad puntual $P(T(\boldsymbol{x}) > T_{\text{crit}})$.

2.7. Estimación por Monte Carlo

La enumeración solo es posible porque N_b es pequeño. Un plano más detallado, por ejemplo con 64 unidades, tendría 2^{64} configuraciones; en ese caso se usa Monte Carlo, cuyo costo no depende de N_b. Por eso se implementó también el estimador

$$\hat{P}_N = \frac{1}{N} \sum_{r=1}^{N} \mathbf{1}\Big\{T_{\text{máx}}(\boldsymbol{A}^{(r)}) > T_{\text{crit}}\Big\}, \tag{13}$$

con $\boldsymbol{A}^{(r)}$ generados por NORTA. En este trabajo cumple dos funciones: verificar el muestreador contra el valor exacto y mostrar cómo se comportaría el método en un problema más grande. Su error estándar es $\sqrt{P_{\text{falla}}(1 - P_{\text{falla}})/N}$, de orden $N^{-1/2}$. La incertidumbre se reporta con el intervalo de Wilson [15], que tiene mejor cobertura que el intervalo normal cuando P_{falla} es pequeña [16, 17]:

$$\frac{\hat{P} + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{P}(1-\hat{P})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \qquad z = 1{,}96. \tag{14}$$

Para comparar diseños se usan *números aleatorios comunes* [18]: el mismo ruido base $\boldsymbol{\varepsilon}^{(r)}$ genera las activaciones de todos los diseños y de todos los valores de ℓ. Como los indicadores de falla de dos diseños están positivamente correlacionados, la varianza de la diferencia $\hat{P}^{(1)} - \hat{P}^{(2)}$ disminuye.

2.8. *Errores de discretización*

El valor calculado de P_{falla} tiene dos fuentes de error: la discretización espacial, que afecta a $T_{\text{máx}}(\boldsymbol{a})$, y el muestreo, que solo afecta al estimador de Monte Carlo. La primera se controla repitiendo el cálculo en mallas de 25^2 a 200^2 celdas (Sección 3.6); la segunda, con los intervalos (14). Todas las mallas del chip tienen bordes de bloque alineados con los elementos, de modo que las funciones indicadoras χ_{Ω_i} se representan sin error.

3. Resultados y análisis

La sección comienza verificando el solver y mostrando su generalidad (Sección 3.1); luego presenta los resultados del estudio del chip. Salvo que se indique otra cosa, estos corresponden a la malla de 100×100 celdas, $\ell = 3\,\text{mm}$, los parámetros de las Tablas 1–2 y $N = 2 \times 10^5$ realizaciones de Monte Carlo. El estudio completo, con las cuatro estrategias y todos los barridos, se ejecuta en menos de un minuto en un computador portátil.

3.1. *Verificación y generalidad del solver*

El solver se verifica con el método de las soluciones manufacturadas [19, 20]: se elige una solución exacta T_{ex}, se calculan con SymPy los datos $(q,\ T_\infty,\ g,\ T_D)$ que la hacen solución de (7), y se comprueba que el error disminuye con el orden teórico al refinar la malla. Para elementos P1 y soluciones suaves, el error en norma L^2 es $\mathcal{O}(h^2)$ [9]. Se trata de una *verificación*, es decir, de comprobar que el programa resuelve correctamente las ecuaciones planteadas; la *validación*, que compararía el modelo con mediciones de un chip real, está fuera del alcance de este trabajo [20].

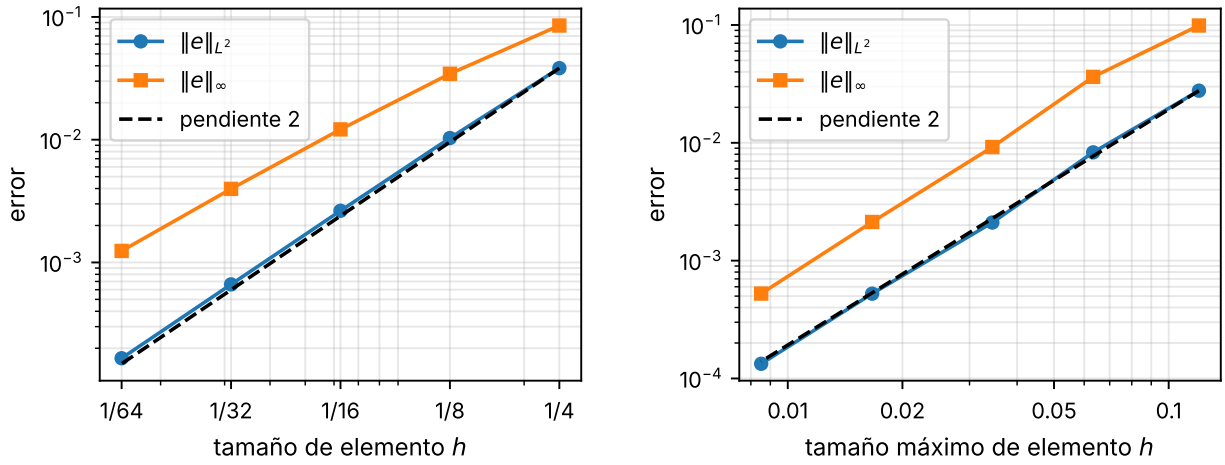
Caso 1: dominio cuadrado y condición de Robin. En $\Omega = (0,1)^2$ con $k = 2$ y $h = 5$ en todo el borde se toma $T_{ex} = 1 + xy^2 + \sin(\pi x)\cos(\pi y/2)$, con $T_\infty = T_{ex} + (k/h)\nabla T_{ex} \cdot \boldsymbol{n}$ en cada lado, y se usan mallas estructuradas.

Caso 2: malla no estructurada, dos materiales y condiciones mixtas. El dominio $(0,1)^2$ tiene un agujero circular de radio 0,15 centrado en (0,75, 0,5) y dos materiales separados por la recta $x = 0{,}5$: $k_1 = 1$, $\beta_1 = 0$ a la izquierda y $k_2 = 4$, $\beta_2 = 3$, $T_{\text{ref}} = 1$ a la derecha. El borde izquierdo es de Dirichlet; los bordes inferior y superior, de Robin con $h = 5$; y el borde derecho y el agujero, de Neumann. La solución exacta es $T_m = u_m(x)\,(2 + \sin \pi y)$ en cada material, con $u_1 = 1 + x^2$ y $u_2 = \frac{5}{4} + \frac{1}{4}\left(x - \frac{1}{2}\right) + \frac{1}{2}\left[1 - \cos 2\pi\left(x - \frac{1}{2}\right)\right]$, elegidas para que la temperatura y el flujo normal $k\,\partial_x T$ sean continuos en la interfaz, lo que se comprueba simbólicamente. Las mallas, generadas con Gmsh, tienen entre 150 y 28.019 nodos (Figura 3). Este caso pone a prueba a la vez la lectura de mallas, los materiales por región, el término de reacción y los tres tipos de frontera.

Tabla 3: Error y orden de convergencia observado $p = \log(e_{k-1}/e_k)/\log(h_{k-1}/h_k)$ en los dos casos de verificación. En el caso 2, h es la arista más larga de la malla.

Caso 1 (estructurada, Robin)					Caso 2 (no estructurada, mixta)				
h	$\|e\|_{L^2}$	p	$\|e\|_\infty$	p	h	$\|e\|_{L^2}$	p	$\|e\|_\infty$	p
1/4	$3{,}83 \times 10^{-2}$	–	$8{,}51 \times 10^{-2}$	–	0,120	$2{,}77 \times 10^{-2}$	–	$9{,}90 \times 10^{-2}$	–
1/8	$1{,}03 \times 10^{-2}$	1,89	$3{,}44 \times 10^{-2}$	1,31	0,063	$8{,}30 \times 10^{-3}$	1,89	$3{,}63 \times 10^{-2}$	1,57
1/16	$2{,}64 \times 10^{-3}$	1,97	$1{,}21 \times 10^{-2}$	1,50	0,034	$2{,}11 \times 10^{-3}$	2,25	$9{,}20 \times 10^{-3}$	2,25
1/32	$6{,}62 \times 10^{-4}$	1,99	$3{,}98 \times 10^{-3}$	1,61	0,017	$5{,}24 \times 10^{-4}$	1,91	$2{,}13 \times 10^{-3}$	2,02
1/64	$1{,}66 \times 10^{-4}$	2,00	$1{,}24 \times 10^{-3}$	1,68	0,0085	$1{,}33 \times 10^{-4}$	2,04	$5{,}25 \times 10^{-4}$	2,09

La Tabla 3 y la Figura 2 muestran que el error en L^2 converge con orden 2 en ambos casos, como predice la teoría. En el caso 1, el error máximo nodal converge con un orden algo menor (1,68),



(a) Caso 1 **(b)** Caso 2

Figura 2: Convergencia del error al refinar la malla en los dos casos de verificación.

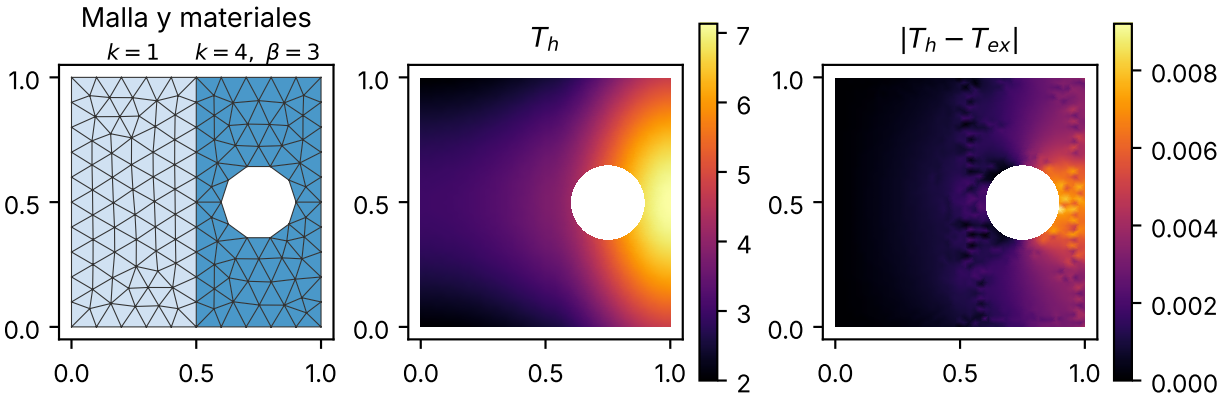


Figura 3: Caso de verificación 2: malla no estructurada más gruesa con los dos materiales, solución de elementos finitos y error absoluto en la malla de 1870 nodos.

consistente con la cota $\mathcal{O}(h^2|\ln h|)$ en norma infinito; el error se concentra en las esquinas, donde se encuentran dos lados con datos de Robin distintos. En el caso 2, el orden observado fluctúa entre 1,9 y 2,3 porque las mallas no estructuradas no se refinan de forma perfectamente uniforme, pero la tendencia es claramente de orden 2. El agujero se aproxima con un polígono, cuyo error geométrico también es $\mathcal{O}(h^2)$ y no degrada la convergencia. Las pruebas automáticas del repositorio incluyen además una *prueba de parche*, que comprueba que una solución lineal se reproduce hasta el error de redondeo, y una prueba de la propiedad de superposición (11).

Para mostrar la generalidad del programa fuera del problema del chip, la Figura 4 resuelve un soporte de aluminio en forma de L ($k = 205\,\mathrm{W/(m\,K)}$) con un inserto de cobre ($k = 400\,\mathrm{W/(m\,K)}$) y un canal circular de refrigeración. La base está en contacto con una pieza a 90 °C (Dirichlet), el canal transporta agua a 20 °C con $h = 2000\,\mathrm{W/(m^2\,K)}$ (Robin), el tope recibe un flujo de $5\,\mathrm{kW/m^2}$ (Neumann) y el resto intercambia calor con el aire. Este problema se resuelve solo cambiando la malla y el archivo JSON.

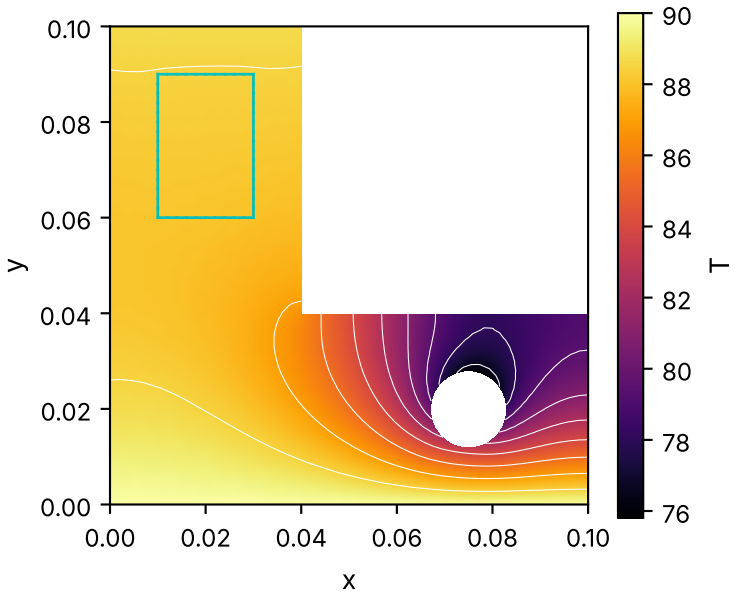


Figura 4: Ejemplo de generalidad: soporte de aluminio en L con un inserto de cobre (contorno cian) y un canal de refrigeración, resuelto a partir de archivos de entrada (1986 nodos). Se combinan condiciones de Dirichlet, Neumann y Robin.

3.2. Comparación de las estrategias de disipación

La Tabla 4 resume los resultados. La primera observación es que, con la misma capacidad total de enfriamiento, la probabilidad de falla varía en un factor de 18 según cómo se distribuya el disipador: de 0,042 con el diseño según potencia pico a 0,755 con el diseño centrado. La ubicación del enfriamiento importa tanto como su cantidad, en coherencia con la longitud de decaimiento $\lambda \approx 1{,}9\,\mathrm{mm}$ de la Sección 2.1: el calor se evacúa cerca de donde se genera. El diseño centrado es el peor porque concentra el enfriamiento sobre la caché, que disipa poco, y deja la mayor parte de los núcleos y de la GPU con la mitad del coeficiente medio (Figura 1). Las estimaciones de Monte Carlo coinciden con los valores exactos dentro de sus intervalos de confianza en los cuatro diseños, lo que verifica la implementación del muestreador NORTA. Las frecuencias de activación simuladas reproducen las p_i objetivo con un error no mayor que 0,003, y la correlación medida entre dos núcleos vecinos (0,295) concuerda con la predicha por (5) (0,297).

Tabla 4: Resultados de las cuatro estrategias ($\ell = 3\,\text{mm}$). P_{falla} exacta por enumeración (12); Monte Carlo con $N = 2 \times 10^5$ e intervalo de Wilson al 95 %. La última columna es el análisis determinista de peor caso.

Estrategia	P_{falla} exacta	$\hat{P}_N$ [IC 95 %]	$\mathbb{E}[T_{\text{máx}}]$ [°C]	Config. que fallan	$T_{\text{máx}}$ todo encendido [°C]
Uniforme	0,451	0,450 [0,448, 0,453]	93,0	96/256	111,9
Centrado	0,755	0,755 [0,753, 0,756]	107,8	217/256	131,6
Según potencia media	0,295	0,296 [0,294, 0,298]	86,9	127/256	106,1
Según potencia pico	0,042	0,042 [0,041, 0,043]	86,4	2/256	100,5

La última columna de la Tabla 4 muestra la limitación del análisis de peor caso. Con todos los bloques encendidos, las cuatro estrategias superan $T_{\text{crit}} = 100\,°\text{C}$, de modo que un criterio determinista las declararía a todas inaceptables y no las podría ordenar. El diseño según potencia pico supera el límite por apenas 0,5 °C y solo en 2 de las 256 configuraciones, lo que en la práctica corresponde a una probabilidad de falla de 4 %, frente a 45 % del diseño uniforme. La diferencia entre ambos, estimada con números aleatorios comunes, es $0{,}409 \pm 0{,}002$.

El resultado más llamativo es la comparación entre los diseños según potencia media y según potencia pico. Sus temperaturas máximas esperadas son casi iguales (86,9 °C frente a 86,4 °C), pero sus probabilidades de falla difieren en un factor de siete. La causa está en la GPU: tiene la segunda mayor densidad de potencia, pero se usa poco ($p = 0{,}3$), así que el diseño según potencia media le asigna poco enfriamiento. En ese diseño fallan 127 de las 128 configuraciones con la GPU encendida y ninguna con la GPU apagada; por eso su P_{falla} es prácticamente igual a $p_{\text{GPU}} = 0{,}3$. Diseñar para el promedio optimiza la temperatura típica pero deja sin protección los eventos poco frecuentes de alta potencia, y la temperatura esperada, por sí sola, no revela ese riesgo.

3.3. Efecto de la correlación entre bloques

La Figura 5 muestra P_{falla} en función de la longitud de correlación. El efecto de la correlación depende del diseño, y se entiende a partir de los casos límite. Cuando $\ell \to \infty$, todas las variables latentes son iguales y los bloques se encienden en un orden fijo, de mayor a menor p_i, de modo que solo quedan $N_b + 1$ configuraciones posibles, anidadas entre sí. La probabilidad de falla tiende entonces a la probabilidad de alcanzar la primera configuración de esa secuencia que falla. Por eso las curvas se aproximan entre sí (entre 0,27 y 0,58 con $\ell = 50\,\text{mm}$) y las diferencias entre diseños se reducen.

- En el **diseño según potencia pico**, la falla es un evento raro que exige muchos bloques encendidos a la vez. La correlación hace más probables esas coincidencias: P_{falla} pasa de 0,008 con bloques independientes a 0,042 con $\ell = 3\,\text{mm}$ y a 0,27 con $\ell = 50\,\text{mm}$, es decir, aumenta más de 30 veces.
- En el **diseño centrado**, la falla es común y basta con pocos bloques encendidos. La correlación aumenta la probabilidad de que haya pocos bloques encendidos, y P_{falla} disminuye de 0,80 a 0,58.
- En el **diseño según potencia media**, P_{falla} casi no cambia, porque depende solo de que la GPU esté encendida, y la probabilidad marginal de un bloque no depende de ℓ.

La implicación práctica es importante: suponer bloques independientes, que es la hipótesis más simple, subestima en un orden de magnitud el riesgo del mejor diseño si la carga real es correlacionada, como ocurre con programas paralelos que usan todos los núcleos al mismo tiempo.

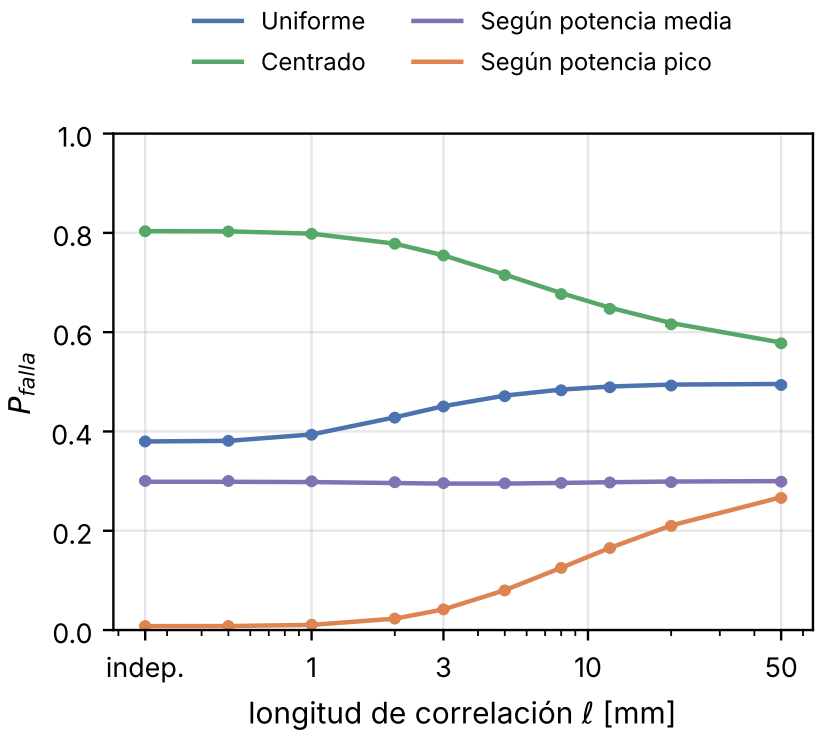
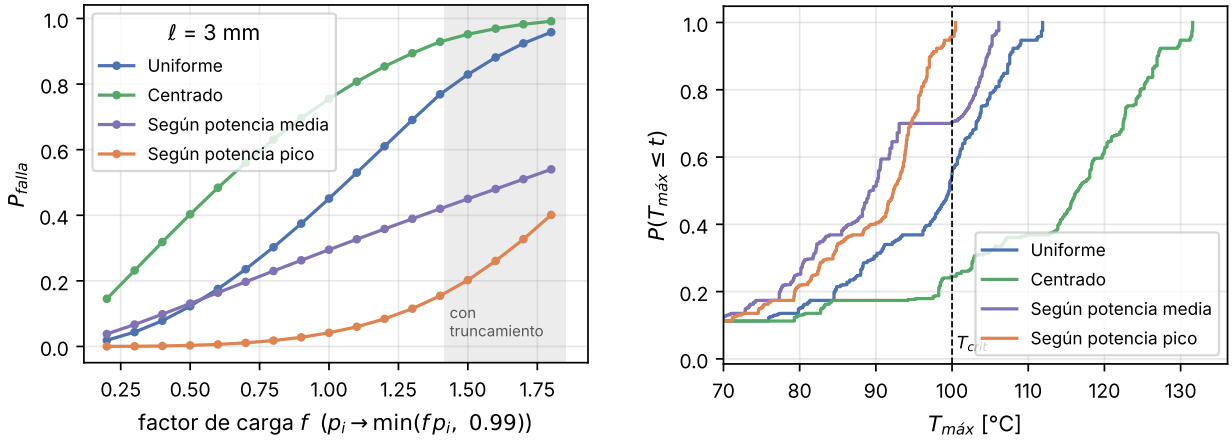


Figura 5: Probabilidad de falla en función de la longitud de correlación ℓ. Las líneas son el valor exacto y los puntos la estimación de Monte Carlo con intervalos de Wilson al 95 % (más pequeños que los marcadores). "indep." corresponde a $\ell = 0$.



(a) Sensibilidad al nivel de carga

(b) Distribución de $T_{\text{máx}}$

Figura 6: (a) P_{falla} cuando todas las probabilidades de activación se escalan por un factor de carga f, con $p_i \to \text{mín}(f\,p_i,\ 0{,}99)$. La zona sombreada ($f > 1{,}41$) indica que alguna probabilidad está truncada. (b) Función de distribución acumulada de la temperatura máxima para $f = 1$; la línea discontinua es T_{crit}.

3.4. Sensibilidad al nivel de carga y distribución de la temperatura máxima

La Figura 6a escala todas las probabilidades de activación por un factor f, que representa usos más livianos ($f < 1$) o más intensos ($f > 1$) del chip. Como una probabilidad no puede superar 1, las probabilidades escaladas se truncan:

$$p'_i = \mín(f\, p_i,\ 0{,}99)\,. \tag{15}$$

El tope es 0,99 y no 1 porque con $p'_i = 1$ el umbral de (4) sería $c_i = \Phi^{-1}(0) = -\infty$: el bloque dejaría de ser aleatorio y las probabilidades de los ortantes quedarían mal definidas. En el rango estudiado ($f \leq 1{,}8$) el truncamiento solo afecta a dos bloques: la caché L3 ($p = 0{,}7$), que se satura a partir de $f \approx 1{,}41$, y el controlador de memoria ($p = 0{,}6$), a partir de $f = 1{,}65$. Los núcleos, la GPU y la E/S nunca se saturan. Por tanto, para $f > 1{,}41$ el escalamiento deja de ser uniforme y la parte derecha de la figura debe leerse con esa salvedad; las conclusiones se apoyan en el rango $f \leq 1{,}4$, donde no hay truncamiento. El diseño según potencia pico es el mejor en todo el rango, pero el orden entre los otros dos no es fijo: con carga baja ($f \lesssim 0{,}55$), el diseño según potencia media es *peor* que el uniforme, y con carga alta es mejor. Esto refuerza la idea central del trabajo: qué diseño conviene depende del uso esperado del chip, y esa dependencia solo se observa con un modelo probabilístico de la carga.

La Figura 6b muestra la distribución completa de $T_{\text{máx}}$. Como hay un número finito de configuraciones, la distribución es escalonada. La curva del diseño según potencia media tiene un salto grande justo por encima de T_{crit}, que corresponde a las configuraciones con la GPU encendida; la del diseño según potencia pico queda casi entera a la izquierda de T_{crit}.

3.5. Distribución espacial del riesgo

La Figura 7 muestra dónde se concentra el riesgo en cada diseño. La desviación estándar de la temperatura es mayor sobre la GPU y los núcleos, los bloques con más potencia y mayor varianza de activación, $p_i(1 - p_i)$. El mapa de probabilidad puntual identifica las zonas que causan las fallas: los núcleos y, en menor medida, la GPU en el diseño uniforme; ambas zonas en el diseño centrado; solo la GPU en el diseño según potencia media; y apenas un punto en el núcleo 2 en el diseño según potencia pico. Estos mapas son útiles para el diseño porque muestran dónde conviene añadir capacidad de enfriamiento y no solo cuánta.

3.6. Convergencia de las estimaciones

La Figura 8a muestra que la estimación de Monte Carlo se estabiliza alrededor del valor exacto y que el ancho de la banda de confianza decrece como $N^{-1/2}$. Para $N = 2 \times 10^5$, el semiancho del intervalo es de 0,001–0,002, muy inferior a las diferencias entre diseños. Los números aleatorios comunes reducen la varianza de la diferencia entre diseños en un factor de 1,2–1,9, según el par comparado y el plano de bloques; la reducción es modesta aquí porque los diseños fallan en configuraciones distintas, lo que debilita la correlación entre sus indicadores.

En cuanto a la malla (Figura 8b), la temperatura máxima con todos los bloques encendidos cambia menos de 0,02 °C entre las mallas de 100^2 y 200^2 celdas. P_{falla} es una función discontinua de la malla, porque una configuración con $T_{\text{máx}}$ muy cercana a T_{crit} puede cambiar de lado al refinar. Así ocurre en el diseño uniforme entre 25^2 y 50^2 celdas, donde P_{falla} pasa de 0,439 a 0,451. A partir de la malla de 50^2, P_{falla} no cambia en ninguno de los cuatro diseños, de modo que el error de discretización es despreciable frente al de muestreo en la malla de 100^2 usada.

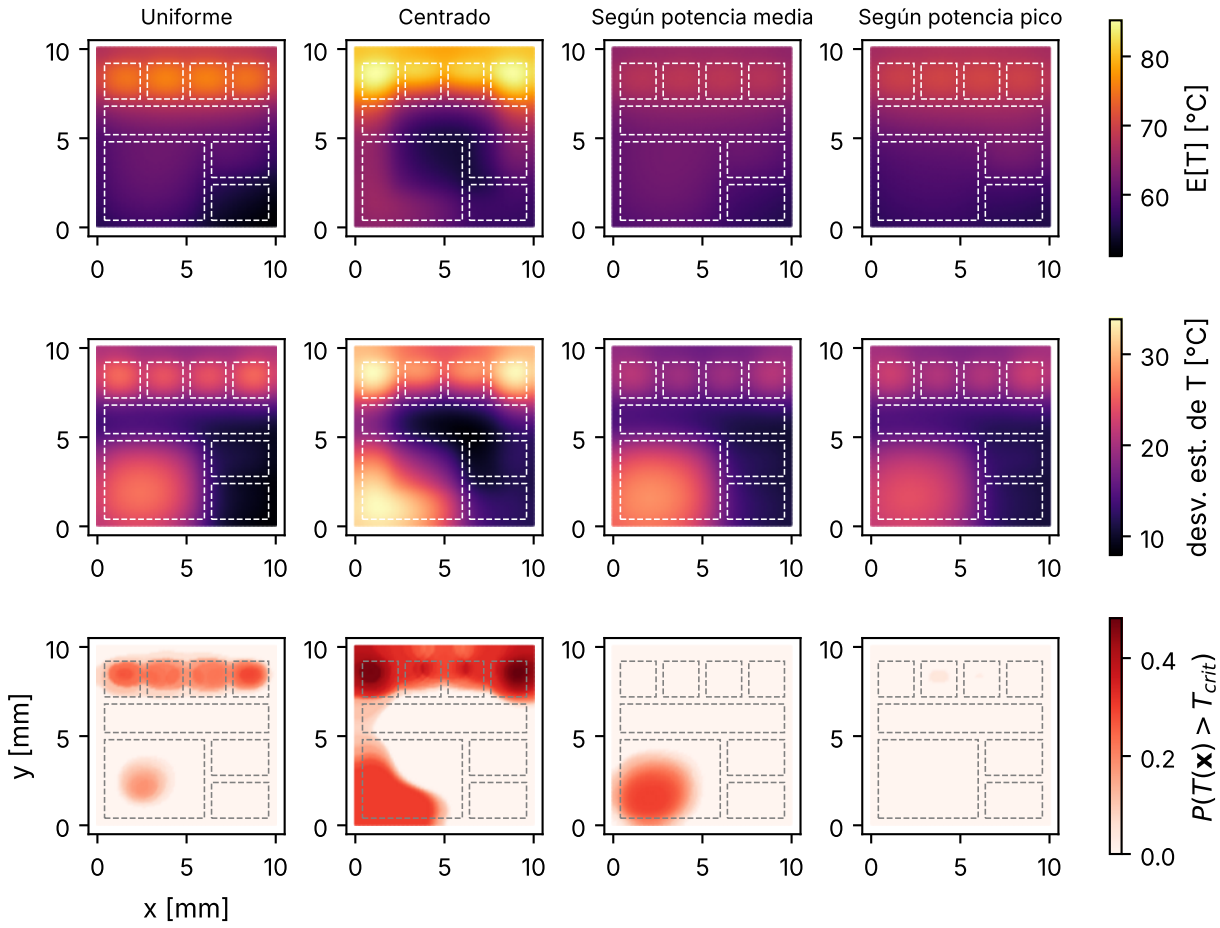
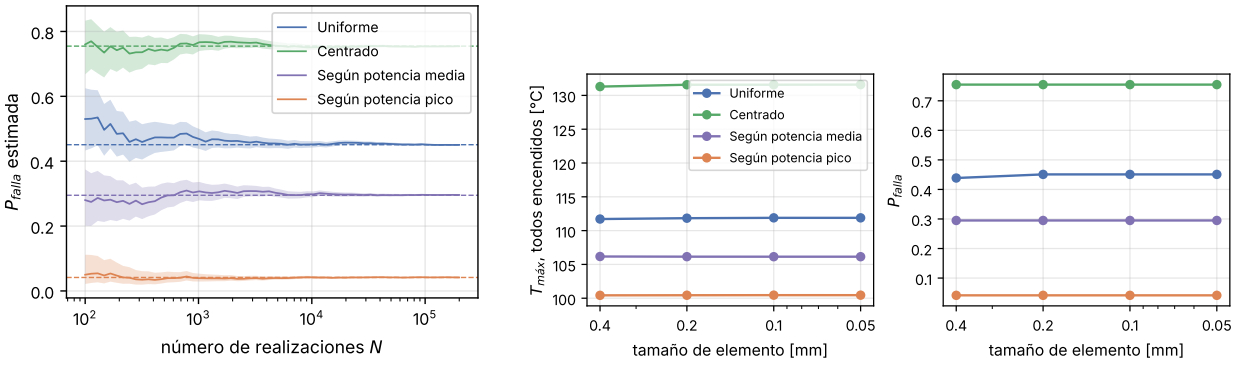


Figura 7: Campos estadísticos exactos para $\ell = 3\,\text{mm}$: temperatura media (arriba), desviación estándar de la temperatura (centro) y probabilidad puntual $P(T(\boldsymbol{x}) > T_{\text{crit}})$ (abajo).



(a) Monte Carlo **(b)** Malla

Figura 8: (a) Estimación de Monte Carlo en función del número de realizaciones, con bandas de Wilson al 95 %; las líneas discontinuas son los valores exactos. (b) $T_{\text{máx}}$ con todos los bloques encendidos y P_{falla} en función del tamaño de elemento.

3.7. Caso idealizado de bloques iguales

Como referencia, el estudio se repitió con el plano de la entrega de avance: nueve bloques iguales de 2 mm, de 13 W cada uno y con $p = 0{,}5$. En ese caso los diseños según potencia media y según potencia pico coinciden exactamente, porque todos los bloques tienen la misma potencia y la misma probabilidad, y se obtiene $P_{\text{falla}} = 0{,}168$ frente a 0,449 del diseño uniforme. La tendencia con ℓ es la misma: P_{falla} crece de 0,049 a 0,42 en el mejor diseño. El caso idealizado reproduce el efecto de la correlación, pero no puede mostrar la diferencia entre diseñar para la media y para el pico, que aparece solo cuando los bloques son heterogéneos. Esto justifica el uso de un plano realista.

4. Conclusiones

Se desarrolló un solver propio de elementos finitos lineales para la ecuación de Helmholtz modificada con varios materiales y condiciones de Dirichlet, Neumann y Robin, configurable mediante archivos de entrada. Se verificó con dos soluciones manufacturadas, una de ellas en mallas no estructuradas con interfaz entre materiales y condiciones mixtas, alcanzando el orden 2 esperado en norma L^2. Con él se estudió el riesgo de sobrecalentamiento de un chip cuya carga de trabajo es aleatoria y correlacionada. Las principales conclusiones son:

1. Con la misma capacidad total de enfriamiento, la distribución espacial del disipador cambia la probabilidad de falla entre 0,04 y 0,75. Dónde se enfría importa tanto como cuánto se enfría.
2. El análisis de peor caso no permite comparar estrategias: las cuatro superan el límite con todos los bloques encendidos, aunque sus probabilidades de falla difieren en un factor de 18.
3. Diseñar según la potencia media produce casi la misma temperatura máxima esperada que diseñar según la potencia pico, pero una probabilidad de falla siete veces mayor, porque desprotege los bloques potentes de uso poco frecuente. La temperatura esperada no es un buen indicador de riesgo.
4. La correlación espacial de la carga aumenta el riesgo de los buenos diseños, más de 30 veces en el mejor caso estudiado, y reduce el de los malos. Suponer independencia puede subestimar gravemente el riesgo real.
5. El orden entre estrategias depende del nivel de carga, por lo que el diseño del disipador debe tener en cuenta el uso esperado del chip.

Desde el punto de vista numérico, la linealidad del problema permitió reducir cada realización a una suma de vectores y calcular la probabilidad de falla de forma exacta por enumeración. Monte Carlo, verificado contra ese valor exacto, queda como la herramienta para planos con muchos más bloques.

Limitaciones y trabajo futuro. El modelo es bidimensional y estacionario: supone que cada configuración de actividad dura más que la constante de tiempo térmica del chip, y resume todo el camino hacia el aire en un único coeficiente h_{sup}. Las potencias y probabilidades de activación son supuestos representativos, no mediciones de un procesador concreto, y la correlación solo depende de la distancia, cuando en la práctica también depende de la función de los bloques. La cópula gaussiana no tiene dependencia en las colas, por lo que podría subestimar la frecuencia de activaciones extremas simultáneas. Como trabajo futuro se proponen: un modelo transitorio con la ecuación del calor, el uso de trazas de potencia medidas para calibrar p_i y Σ, la optimización de h_{sup} para minimizar P_{falla} con la capacidad total fija, y la validación del modelo con mediciones experimentales o con un simulador de referencia como HotSpot [5].

Disponibilidad del código

El código fuente, las mallas, los archivos de entrada, las pruebas automáticas y los scripts que reproducen todas las figuras y tablas están disponibles en https://github.com/marioarroyaveaguirre2004-boop/proyecto-metodos-edp-Grupo6. Cada figura se regenera con un solo comando y semilla fija.

Declaración sobre el uso de inteligencia artificial

Durante este trabajo los autores usaron un asistente de inteligencia artificial generativa (Claude, de Anthropic) como apoyo en el desarrollo y la depuración de partes del código, en la generación de figuras y en la redacción de borradores del texto. Los autores revisaron, ejecutaron y verificaron todo el código y los resultados, y editaron el contenido. Asumen total responsabilidad por el contenido de esta publicación.

Referencias

[1] K. Skadron, M. R. Stan, W. Huang, S. Velusamy, K. Sankaranarayanan, D. Tarjan, Temperature-aware microarchitecture, in: Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA), IEEE, 2003, pp. 2–13. doi:10.1109/ISCA.2003.1206984.

[2] M. Pedram, S. Nazarian, Thermal modeling, analysis, and management in VLSI circuits: Principles and methods, Proceedings of the IEEE 94 (8) (2006) 1487–1501. doi:10.1109/JPROC.2006.879797.

[3] Intel Corporation, Information about temperature for intel® processors, https://www.intel.com/content/www/us/en/support/articles/000005597/processors.html, consultado el 6 de noviembre de 2026 (2026).

[4] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam, D. Burger, Dark silicon and the end of multicore scaling, in: Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA), ACM, 2011, pp. 365–376. doi:10.1145/2000064.2000108.

[5] W. Huang, S. Ghosh, S. Velusamy, K. Sankaranarayanan, K. Skadron, M. R. Stan, HotSpot: A compact thermal modeling methodology for early-stage VLSI design, IEEE Transactions on Very Large Scale Integration (VLSI) Systems 14 (5) (2006) 501–513. doi:10.1109/TVLSI.2006.876103.

[6] F. P. Incropera, D. P. DeWitt, T. L. Bergman, A. S. Lavine, Fundamentals of Heat and Mass Transfer, 6th Edition, John Wiley & Sons, Hoboken, NJ, 2007.

[7] C. J. Glassbrenner, G. A. Slack, Thermal conductivity of silicon and germanium from 3°K to the melting point, Physical Review 134 (4A) (1964) A1058–A1069. doi:10.1103/PhysRev.134.A1058.

[8] M. C. Cario, B. L. Nelson, Modeling and generating random vectors with arbitrary marginal distributions and correlation matrix, Tech. rep., Department of Industrial Engineering and Management Sciences, Northwestern University, Evanston, IL (1997).

[9] M. G. Larson, F. Bengzon, The Finite Element Method: Theory, Implementation, and Applications, Vol. 10 of Texts in Computational Science and Engineering, Springer, Berlin, Heidelberg, 2013. doi:10.1007/978-3-642-33287-6.

[10] A. Meurer, C. P. Smith, M. Paprocki, et al., SymPy: symbolic computing in Python, PeerJ Computer Science 3 (2017) e103. doi:10.7717/peerj-cs.103.

[11] P. Virtanen, R. Gommers, T. E. Oliphant, et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.

[12] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities, International Journal for Numerical Methods in Engineering 79 (11) (2009) 1309–1331. doi:10.1002/nme.2579.

[13] C. R. Harris, K. J. Millman, S. J. van der Walt, et al., Array programming with NumPy, Nature 585 (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[14] A. Genz, Numerical computation of multivariate normal probabilities, Journal of Computational and Graphical Statistics 1 (2) (1992) 141–149. doi:10.1080/10618600.1992.10477010.

[15] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[16] A. Agresti, B. A. Coull, Approximate is better than "exact" for interval estimation of binomial proportions, The American Statistician 52 (2) (1998) 119–126. doi:10.1080/00031305.1998.10480550.

[17] L. D. Brown, T. T. Cai, A. DasGupta, Interval estimation for a binomial proportion, Statistical Science 16 (2) (2001) 101–133. doi:10.1214/ss/1009213286.

[18] P. Glasserman, Monte Carlo Methods in Financial Engineering, Springer, New York, 2003. doi:10.1007/978-0-387-21617-1.

[19] P. J. Roache, Code verification by the method of manufactured solutions, Journal of Fluids Engineering 124 (1) (2002) 4–10. doi:10.1115/1.1436090.

[20] W. L. Oberkampf, C. J. Roy, Verification and Validation in Scientific Computing, Cambridge University Press, Cambridge, 2010. doi:10.1017/CBO9780511760396.